

Microservicios, Persistencia NoSQL y Atributos de Calidad

CG

May 18, 2026

1. En la arquitectura de FinCore, un desarrollador propone implementar el patrón **Microservice Chassis**. cual es el principal beneficio arquitectonico que se busca con esta decisión?

Reducir el tiempo de comercialización y asegurar la consistencia transversal al **encapsular** como logs, seguridad y métricas en una plantilla reutilizable.

2. al aplicar el patrón **Externalized configuration**, que impacto se genera directamente sobre los ASRs del sistema?

Favorece la modificabilidad al permitir cambios de parámetros en tiempo de ejecución o arranque sin recompilar el artefacto, pero puede añadir un riesgo de seguridad si no se cifran los almacenes centralizados.

3. un Message Broker con garantía de entrega 'At least once' duplica un mensaje debido a un fallo de confirmación de red. Si el consumidor implementa el patrón Idempotent Consumer utilizando Redis como almacén de IDs distribuidos, ¿qué acción toma el sistema?

Verifica si el ID de mensaje ya existe en Redis; al encontrarlo, descarta el mensaje duplicado e impide la **re-ejecución** de la lógica de negocio.

4. Considera la estrategia de despliegue 'Multiple Service Instances per Host'. ¿Cuál es su principal debilidad en un entorno de alta concurrencia como el de BITE.co?

La falta de aislamiento de recursos, lo que permite que una sola instancia sature la CPU o memoria del host, degradando la disponibilidad de todos los demás servicios compartidos. *compartir el mismo espacio de sistema operativo significa que no hay limites estrictos de hardware por proceso, induciendo contención de recursos*

5. En la estrategia de despliegue 'Service Instance per VM', ¿cuál es el trade-off directo en comparación con el uso de contenedores?

Se maximiza el aislamiento de fallas y seguridad a nivel de hipervisor, pero a costa de un mayor consumo de recursos y tiempos de aprovisionamiento elevados. *cada VM arranca un SO completo, garantizando fronteras rígidas de hardware y red, pero penalizando el tamaño del artefacto y su velocidad de despliegue*

6. El documento del Grupo 1 describe la estrategia 'Service Instance per Container'. ¿Por qué la seguridad se evalúa con una calificación media/baja (B) en su matriz de trade-offs?

porque los contenedores comparten el mismo kernel del sistema operativo del host, lo que amplía la superficie de ataque si un proceso logra romper el aislamiento del espacio de nombres(namespace). *a diferencia de las VMs con kernels separados, una vulnerabilidad a nivel de kernel en el host puede comprometer potencialmente a todos los contenedores residentes*

7. Bajo un esquema de 'Serverless Deployment' (ej. AWS Lambda), ¿cuál de las siguientes situaciones describe el fenómeno conocido como 'Cold Start'?

La latencia adicional que ocurre cuando llega una petición y la plataforma debe aprovisionar un contenedor efímero, descargar el código e iniciar el entorno de ejecución desde cero. *al no haber servidores encendidos continuamente, la primera llamada sufre un retraso mientras la plataforma Serverless orquesta la infraestructura base*

8. En el patrón de descubrimiento 'Client-side Discovery', ¿cuál es el rol exacto del cliente que desea invocar a otro servicio?

Consultar activamente al service registry las ubicaciones de red disponibles, aplicar localmente un algoritmo de balanceo de carga y direccionar la petición directamente a la instancia seleccionada. *en el enfoque de cliente, la inteligencia de consulta de catálogo y la distribución del tráfico reside en el proceso que origina la llamada*

9. En un esquema de 'Server-side Discovery', ¿qué atributo de calidad se ve amenazado al introducir un enrutador intermedio (como un Balanceador de Carga)?

La disponibilidad, puesto que el balanceador de carga centralizado se convierte en un punto único de falla, si no se configura con redun-

dancia activa. *toda comunicación obligatoriamente pasa por este nodo intermedio, si colapsa, la comunicación interservicios se interrumpe por completo*

10. El equipo de arquitectura debate entre 'Self Registration' y '3rd Party Registration'. ¿En qué escenario es técnicamente indispensable optar por '3rd Party Registration'?

cuando se cuenta con un entorno poliglota complejo y servicios legados de caja negra cuyo código fuente no puede modificarse para embeber lógica de infraestructura de red.

11. En el patrón 'API Composition', si el compositor orquesta llamadas síncronas en paralelo a tres servicios independientes para armar una vista unificada, ¿cómo se calcula teóricamente el tiempo mínimo de respuesta total de la consulta?

es igual al tiempo de respuesta del microservicio que responda más lento, más la sobrecarga de red y el procesamiento de agregación del propio compositor.

12. La segregación de responsabilidad de comandos y consultas (CQRS) resuelve la contención de recursos en alta concurrencia. Sin embargo, ¿qué costo crítico impone sobre el sistema?

introduce consistencia eventual, lo que significa que las consultas pueden retornar datos transaccionales desactualizados por milisegundos mientras se propaga la sincronización asíncrona.

13. En el patrón 'Event Sourcing', ¿cómo se determina el estado actual de una entidad de negocio en un momento dado?

Recuperando la secuencia cronológica completa de eventos mutativos inmutables guardados en el Event Store reproduciendolos secuencialmente en memoria (rehidratación)

14. Para mitigar la degradación de desempeño que causa reproducir miles de eventos históricos en Event Sourcing, ¿qué mecanismo técnico se debe incorporar?

La generación periódica de instantáneas (snapshots) que guardan el estado consolidado cada cierto número de eventos, permitiendo rehidratar solo los eventos ocurridos después del último Snapshot.

15. De acuerdo con el documento de diseño detallado NoSQL, ¿cuál es la premisa fundamental para modelar datos en bases documentales?

Diseñar las estructuras de los documentos adaptandose estrictamente a los patrones de acceso y consultas de la aplicación, aceptando la desnormalización si favorece el desempeño.

16. En una colección de productos de comercio electrónico con propiedades altamente heterogéneas, se aplica el 'Attribute Pattern'. ¿Cuál es el cambio estructural que sufre el documento JSON?

se encapsulan las propiedades variables dentro de un arreglo único de subdocumentos con la estructura fija llave-valor [*"k": "nombre-propiedad", "v": "valor"*]

17. Al utilizar el 'Attribute Pattern' en MongoDB, ¿cuál es el beneficio directo sobre la infraestructura de indexación?

Habilita la creación de un único índice compuesto sobre las propiedades del arreglo, evitando la explosión de índices individuales pesados.

18. El patrón NoSQL 'Schema Versioning' dictamina que los documentos antiguos y nuevos coexistan en la misma colección de producción. ¿Cómo maneja la aplicación la disparidad de estructuras en tiempo de lectura?

La capa de lógica del microservicio lee el campo (schema version) y ejecuta condicionales o fabricas polimorficas para adaptar o rellenar dinámicamente los campos faltantes según la version leida.

19. En un sistema de blogs de alta lectura, un post puede recibir millones de comentarios. Si decidimos embeber todos los comentarios dentro del documento del post, ¿qué riesgo técnico directo identificamos según el 'Subset Pattern'?

Superar el tamaño limite físico del documento JSON/BSON del motor o saturar el buffer de cache en memoria RAM cargando miles de datos históricos irrelevantes para la vista inicial.

20. El 'Subset Pattern' soluciona el problema de los arreglos con crecimiento desmedido. ¿Cuál es la estrategia de diseño detallada que propone?

Mantener embebido dentro del documento principal únicamente un numero limitado y fijo de los elementos mas recientes o leídos, y trasladar el resto de los registros históricos a una colección externa paginada.

21. Un sistema financiero calcula el saldo consolidado de un cliente sumando millones de transacciones históricas en tiempo de ejecución cada vez que abre la app. Al aplicar 'Computed Pattern', ¿cómo se optimiza esta operación?

Se desplaza el costo computacional al momento de la escritura: cada vez que se genera una transacción, el microservicio calcula el nuevo saldo acumulado y lo almacena directamente en un campo dedicado en el documento del cliente.

22. Para combatir ataques de Denegación de Servicio (DoS) en el Core Bancario de FinCore, el API Gateway implementa Rate Limiting y Circuit Breakers. ¿Qué tipo de tácticas de Disponibilidad representan?

Tácticas de recuperación y prevención de fallas, al limitar la tasa de entrada de peticiones externas masivas y aislar proactivamente servicios lentos para proteger los hilos del sistema. *impedir que el tráfico desmedido consuma el pool de hilos previene el desfallecimiento general del host y aísla los componentes degradados*